

To build a robust, scalable students grade microservice suite in Scala, you need a modular architecture that separates concerns like grade calculation, report generation, and data ingestion. Here is the blueprint for your microservice suite's requirements, tech stack, and core architecture.

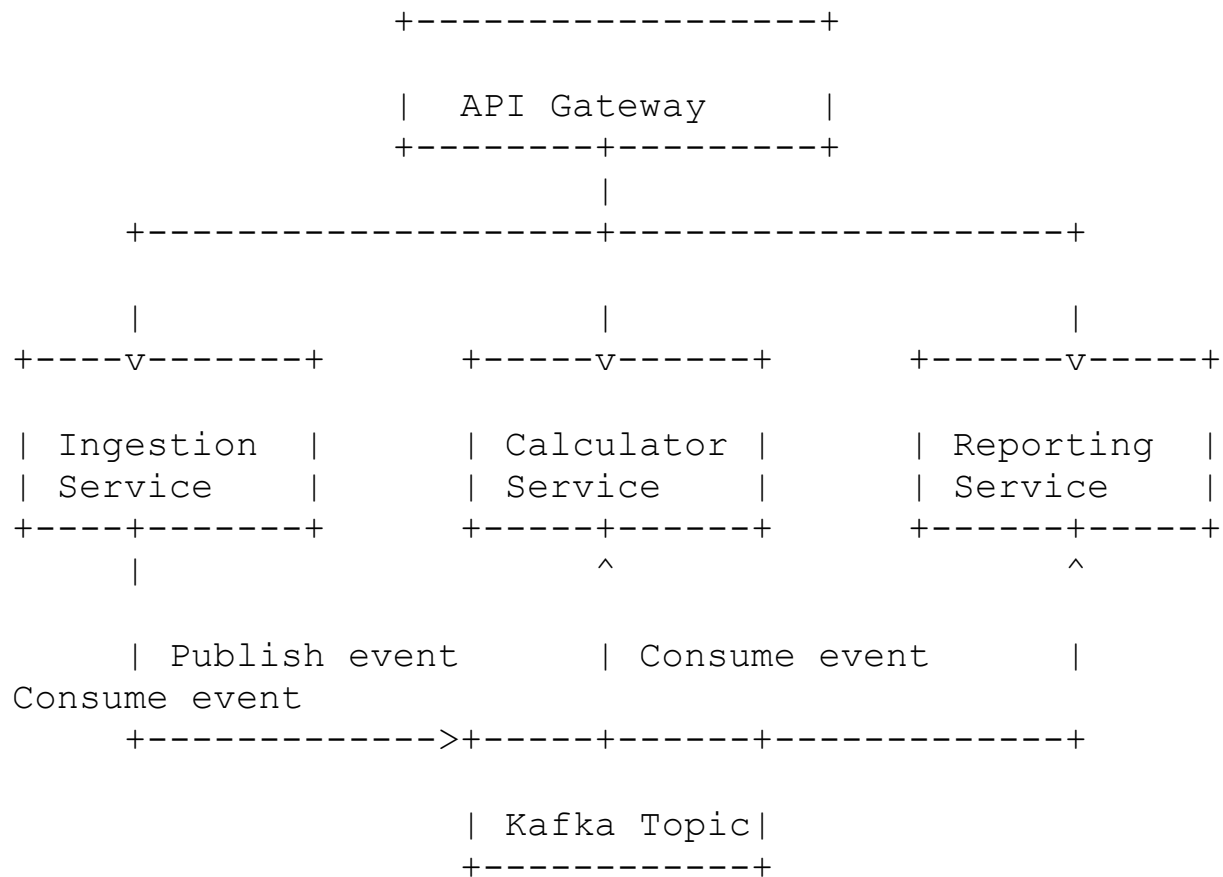
Functional Requirements

- **Grade Ingestion:** Accept individual or bulk assignment scores from teachers.
- **Grade Calculation:** Compute final letter grades, GPAs, and class averages.
- **Report Generation:** Generate digital report cards and transcripts for students.
- **Audit Logging:** Track all grade modifications for security compliance.
- **Access Control:** Restrict endpoints based on roles (Students, Teachers, Admins).

Recommended Tech Stack

- **Language:** Scala 3 (for advanced type safety and clean syntax).
- **Framework:** Typelevel Stack (Cats Effect, Http4s, FS2) or ZIO (ZIO HTTP, ZIO Schema).
- **Database:** PostgreSQL (for ACID compliance) with Skunk or Quill for queries.
- **Messaging:** Apache Kafka (for event-driven communication between services).
- **Testing:** MUnit or ScalaTest with Weaver-test for integration tests. [[1](#), [2](#), [3](#)]

Architecture & Service Boundaries



Core Microservices Breakdown

1. Grade Ingestion Service

- **Purpose:** Handles the entry of raw marks.
- **Endpoints:** `POST /grades/input`, `PUT /grades/{id}`.
- **Data Validation:** Ensures scores fit within valid ranges (e.g., 0-100).
- **Async Output:** Publishes a `GradeSubmitted` event to Kafka.

2. Grade Calculator Service

- **Purpose:** Computes weighted scores, curve adjustments, and GPAs.
- **Trigger:** Listens to the `GradeSubmitted` Kafka event.
- **State:** Maintains a localized cache of grading schemas per course.
- **Async Output:** Publishes a `GradeCalculated` event to Kafka.

3. Reporting & Analytics Service

- **Purpose:** Compiles student transcripts and class metrics.
- **Trigger:** Listens to `GradeCalculated` events to update dashboards.
- **Endpoints:** `GET /students/{id}/report-card`, `GET /classes/{id}/analytics`.
- **Storage:** Read-heavy database optimized for analytical queries (e.g., ClickHouse or a Postgres Read Replica).

Non-Functional Requirements

- **Concurrency:** Use functional effects (Cats Effect Fibers or ZIO Fibers) to handle thousands of concurrent grade submissions smoothly.
- **Resilience:** Implement circuit breakers (via Resilience4s) on external calls to identity management systems.
- **Data Privacy:** Encrypt student data at rest and in transit to comply with FERPA or GDPR regulations.
- **Eventual Consistency:** Ensure your calculator and reporting services use idempotent consumers to handle duplicate Kafka messages safely. [\[1\]](#)

To help tailor this design, tell me:

- Which ecosystem do you prefer? (**ZIO** or **Typelevel/Cats**?)
- How will this handle **bulk uploads**? (Real-time APIs or CSV batch processing?)
- What **hosting environment** are you using? (Kubernetes, AWS, etc.?)